Homework 4
Computer Vision, Spring 2026
Due Date: March 18, 2026
Total Points: 20

runHw4.py will be your main interface for executing and testing your code. Parameters for the different programs or unit tests can also be set in that file. Before submitting, make sure you can run all your programs with the command `python runHw4.py all` with no errors.

Challenge 1: Your task is to develop an "Image Mosaicking App" that stitches a collection of photos into a mosaic. Creating image mosaics and panoramas have become one of the most popular features on smart phones. Outside of the realm of smart phones, similar applications have also been developed to create stunning panoramas seen on gigapan.org. With the concepts and algorithms presented in the Image Alignment lecture, you too can create an image mosaic.

Before we create the mosaicking app, we will create the individual tools required to build it. Each tool is a separate program you need to write and submit.

a. In this part, the goal is to develop a program to calculate homography between a pair of images. We will also develop two additional small programs to help verify whether the calculated homography is correct.

   First, write a program named `computeHomography` that calculates the homography between two sets of corresponding points in two images: `H = computeHomography(src_pts, dest_pts)`.

   A pair of images, related by a homography, are provided to help test your program. You will choose at least four corresponding points manually and use them to compute the homography. **(2 points)**

   Second, write a program named `applyHomography` that applies homography to a set of points:
   `dest_pts = applyHomography(H, test_pts)`.
   Choose a new set of points `test_pts` from the first image and apply the computed homography to them. **(1 points)**

Third, write a program named `showCorrespondence` to annotate the corresponding points in the two images:
`result_img = showCorrespondence(src_img, dest_img, src_pts, dest_pts)`.
Here `src_img` and `dest_img` are two images related by a homography, and `result_img` is an image showing `src_img` and `dest_img` side-by-side, with lines connecting `src_pts` and `dest_pts`. Figure 1 shows an example output of `showCorrespondence`. **(2 points)**

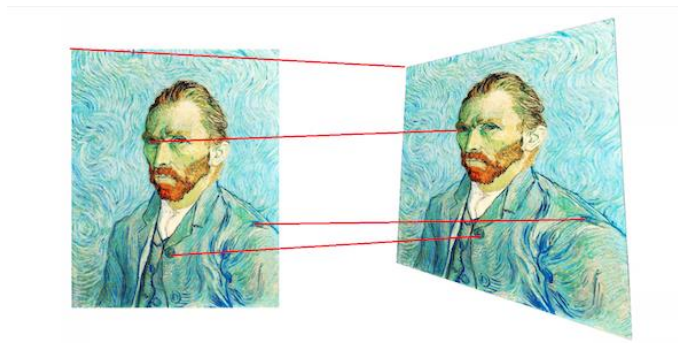


Figure 1

b. Develop a program to warp an image. Use it to warp and paste a portrait of Vincent to the empty billboards in the image shown in Figure 2. To help you get started, a skeleton code has been provided in `challenge1b`.

Before [3] After

Figure 2

First, you need to estimate the homography from the portrait to the target billboard frame. Fill in the missing parts in `challenge1b` where appropriate and use the programs developed earlier to compute a homography.

Second, write a program named `backwardWarpImg` that warps an image based on a homography:
`[mask, dest_img] = backwardWarpImg(src_img, dest_to_src_homography, dest_canvas_width_height)` .
The input argument `src_img` is the source image (the portrait in this case). Because you will implement **backward warping**, `dest_to_src_homography` is the inverse of the homography that maps source to destination. `dest_canvas_width_height` specifies width and height of the destination image.

Now the output arguments. `dest_img` is the output image. In this case, `dest_img` is a warped portrait on a black canvas. `mask` is a binary mask indicating the area of the warped image on the canvas. `mask` is needed because the warped image is often not the final result, but used as an input to other post-processing tasks (in this case, you will post the warped image to another image).

After generating a warped portrait image and its associated mask, superimpose the image on the given billboard image. A reference code is provided to help you perform this task. **(4 points)**

c. Recall the outlier problem discussed in the Image Alignment lecture. Some of the corresponding points, referred to as outliers, computed using automatic interest point detectors may not concur with the actual homography. Write a function named `runRANSAC` to robustly compute homography using RANSAC to address the outlier problem.

`[inliers_id, src_to_dest_H] = runRANSAC(src_pt, dest_pt, ransac_n, ransac_eps)` .
Figures 3 and 4 show the matches between two images, before and after running RANSAC. **(4 points)**

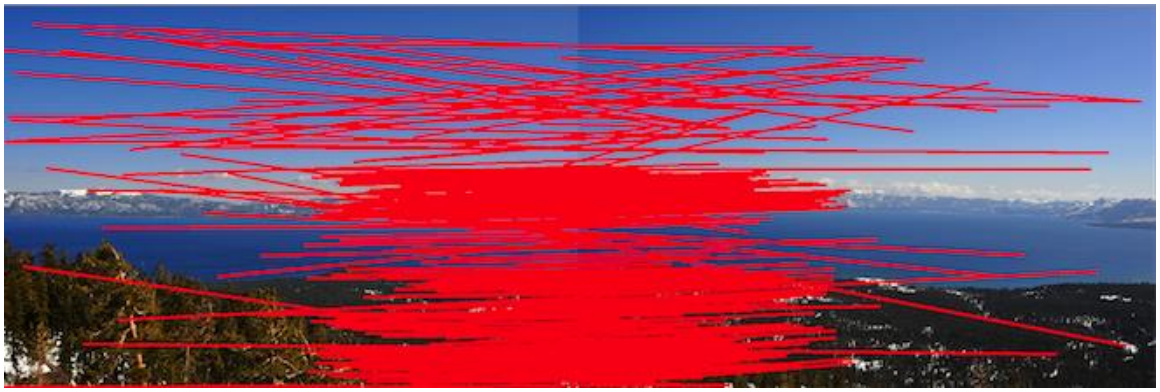
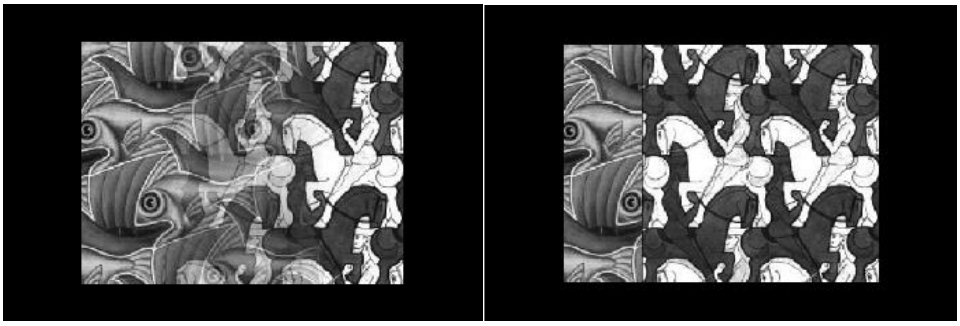


Figure 3

Figure 4

d. Write a program named `blendImagePair` that blends two images into one:
`result = blendImagePair(img1, mask1, img2, mask2, blending_mode)` .
`img1` and `img2` are two input images. `mask1` and `mask2` are the corresponding binary masks indicating the regions of interest.
`blending_mode`, a string, is either `"overlay"` or `"blend"`. In the case of `"overlay"`, copy `img2` over `img1` wherever the `mask2` applies. In the case of `"blend"`, perform weighted blending as discussed in class. Figure 5

shows example outputs (left: blended, right: overlay). **(2 points)**



1 Figure 5

e. Now you have all the tools to build the mosaicking app! Write a program `stitchImg` that stitches the input images into one mosaic.
`stitched_img = stitchImg(img1, img2, …, imgN)`.
Use "`blend`" mode to blend the image when you call `blendImagePair`. **(4 points)**

f. Capture images using your own (cell phone) camera and stitch them to create a mosaic. Note that the mosaic need not be a horizontal panorama. Submit both the captured and stitched images. **(1 point)**

References

[1] MathWorks. Vectorization. [Online]. http://www.mathworks.com/help/matlab/matlab_prog/vectorization.html

[2] MathWorks. Technique for Improving Performance. [Online]. http://www.mathworks.com/help/matlab/matlab_prog/techniques-for-improving-performance.html

[3] Flickr. [Online]. http://www.flickr.com/photos/schmaeche/2545319687/